LAPORAN PRAKTIKUM

MATA KULIAH TEKNOLOGI IOT II

MODUL 7: INTEGRASI BACKEND SERVER LOKAL (DOCKER & MOSQUITTO) DAN RULE ENGINE (NODE-RED)

Oleh:

Nama: Aila Nadianti

NPM : 25782059

Kelas : 3C

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

7.4 Persiapan Praktikum

7.4.1 Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia di meja praktikum Anda (identik dengan perangkat praktikum Modul 6):

1. *Board* NodeMCU ESP8266 (1 unit)
2. Kabel Data Micro-USB (1 unit)
3. *Breadboard* / Papan Proyek (1 unit)
4. Sensor Suhu & Kelembapan DHT11 atau DHT22 (1 unit)
5. LED Indikator 5mm (1 unit)
6. Resistor 220 Ω(1 unit, pembatas arus LED)
7. Kabel *Jumper* tipe *Male-to-Male* dan *Male-to-Female* (secukupnya)
8. Komputer/Laptop pengembang (Windows, macOS, atau Linux) dengan RAM minimal 4 GB.
9. Sumber Jaringan Wi-Fi Lokal (Disarankan menggunakan *Tethering / Mobile Hotspot* dari ponsel di mana laptop dan NodeMCU terhubung ke subnet yang sama).

7.4.2 Persiapan Perangkat Lunak & Lingkungan Docker

1. **Docker Engine & Docker Compose:**
    - Untuk pengguna **Windows / macOS**: Unduh dan pasang **Docker Desktop** dari situs resmi docker.com. Pastikan status Docker Desktop di pojok bilah tugas menunjukkan ikon hijau bertuliskan **Engine Running**.
    - Untuk pengguna **Linux**: Pastikan paket docker dan plugin docker-compose-plugin telah terpasang dan servis Docker aktif (sudo systemctl start docker).
2. **Arduino IDE:**
    - Pastikan pustaka berikut telah terpasang melalui *Library Manager*:
        - PubSubClient oleh Nick O'Leary.
        - ArduinoJson oleh Benoit Blanchon (versi 6.x disarankan).
        - DHT sensor library oleh Adafruit beserta dependensi *Adafruit Unified Sensor*.
3. **Aplikasi Teks Editor:**
    - Visual Studio Code, Notepad++, atau editor teks standar untuk membuat berkas docker-compose.yml dan mosquitto.conf.
4. **Peramban Web Modern:**

- Google Chrome, Mozilla Firefox, atau Microsoft Edge untuk mengakses dasbor Node-RED di port 1880.

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.

Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).
2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:

   iot-server/
   ├── mosquitto/
   │ └── config/

```
D:\iot-server>tree
Folder PATH listing for volume Local Disk
Volume serial number is 4EB9-482D
D:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───nodered
    ├───lib
    │   └───flows
    └───node_modules
```

Gambar 1. Struktur Direktori IoT Server

   Gambar di atas menunjukkan hasil perintah tree pada CMD untuk melihat struktur folder iot-server yang telah dibuat. Terlihat folder mosquitto yang memiliki subfolder config, data, dan log, serta folder nodered yang berisi lib, flows, dan node_modules. Struktur folder ini digunakan untuk memisahkan penyimpanan konfigurasi MQTT Broker dan data Node-RED agar lebih teratur serta mudah dikelola.

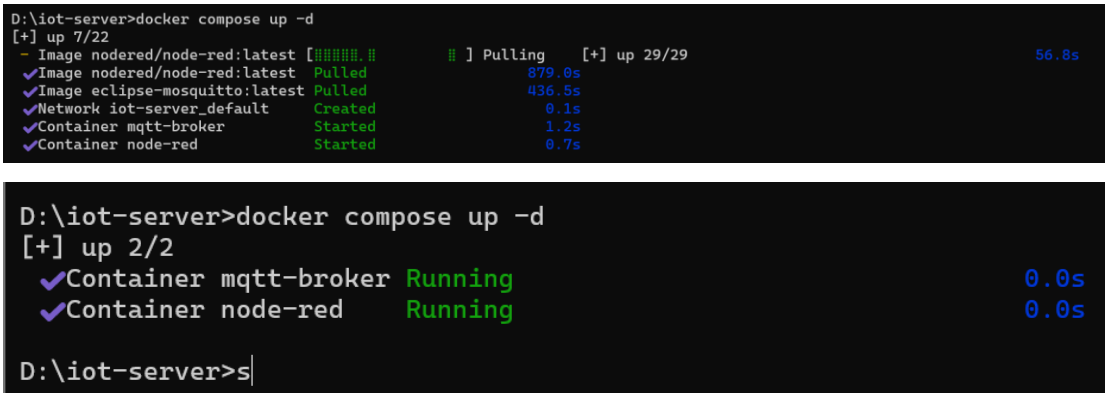
3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama **mosquitto.conf**, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter

4. Kembali ke folder induk iot-server, buat berkas baru bernama **docker-compose.yml**, lalu salin konfigurasi layanan terpadu
5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:

   cd D:\iot-server
6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):

   docker compose up -d

   (Catatan: Pada versi Docker lama, gunakan perintah docker-compose up -d).



Gambar 2. Proses Menjalankan Container Menggunakan Docker Compose

Gambar di atas menunjukkan proses menjalankan perintah docker compose up -d melalui Command Prompt (CMD) pada direktori D:\iot-server. Terlihat bahwa Docker Compose berhasil menjalankan dua container, yaitu mqtt-broker dan node-red, dengan status *Started*. Perintah ini digunakan untuk menjalankan kedua layanan di latar belakang sehingga MQTT Broker dan Node-RED dapat digunakan sebagai server lokal dalam sistem IoT.

Status Started menunjukkan bahwa container telah dijalankan. Untuk memastikan keduanya masih aktif, dilakukan pemeriksaan menggunakan perintah docker ps pada langkah berikutnya.

7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.
8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

   docker ps

   Pastikan kolom STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).

```
D:\iot-server>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS                    PORTS
                                                                 NAMES
e5bdfa48ddd4   nodered/node-red:latest  "./entrypoint.sh"        43 minutes ago   Up 43 minutes (healthy)   0.0.0.0:1880->1880/tcp,
 [::]:1880->1880/tcp                                             node-red
eeb50ab5f7a6   eclipse-mosquitto:latest "/docker-entrypoint.…"   43 minutes ago   Up 43 minutes             0.0.0.0:1883->1883/tcp,
 [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp  mqtt-broker
e9ab957f4e03   wordpress:latest         "docker-entrypoint.s…"   10 days ago      Up About an hour          0.0.0.0:6969->80/tcp, [
::]:6969->80/tcp                                                 wordpress
6fb31ed8ad33   mysql:latest             "docker-entrypoint.s…"   10 days ago      Up About an hour          3306/tcp, 33060/tcp
                                                                 mysql-wp
```

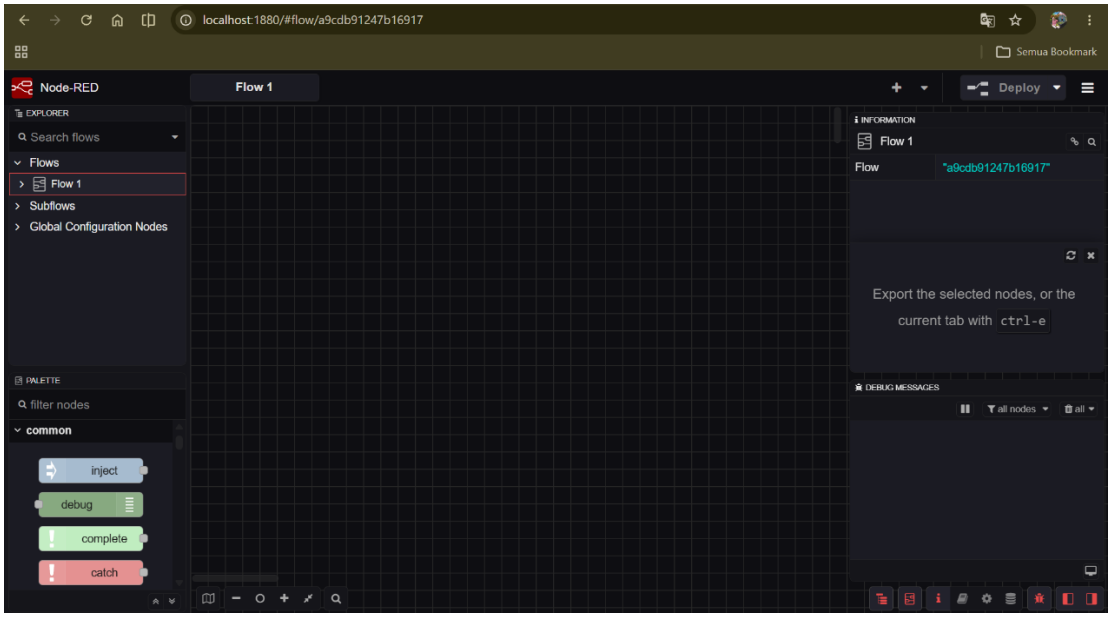
Gambar 3. Hasil Pemeriksaan Status Container Menggunakan Docker PS

Gambar di atas menunjukkan hasil pemeriksaan container Docker menggunakan perintah docker ps melalui Command Prompt (CMD). Terlihat dua container, yaitu mqtt-broker dan node-red, memiliki status *Up*, yang menandakan keduanya sedang berjalan. Pada tampilan tersebut juga terlihat port 1883 dan 9001 untuk MQTT Broker serta port 1880 untuk Node-RED. Hasil ini menunjukkan bahwa kedua layanan telah aktif dan siap digunakan untuk praktikum IoT.

Perintah docker ps digunakan untuk memastikan container masih berjalan setelah dijalankan menggunakan Docker Compose.

9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: **http://localhost:1880**.

   Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!



Gambar 4. Tampilan Awal Antarmuka Node-RED

Gambar di atas menunjukkan tampilan awal Node-RED yang berhasil diakses melalui browser menggunakan alamat http://localhost:1880. Terlihat halaman kerja Node-RED dengan kanvas kosong di bagian tengah, daftar node di sebelah kiri, dan panel informasi di sebelah kanan. Tampilan ini menunjukkan bahwa layanan Node-RED sudah berjalan

dan dapat diakses untuk membuat alur pengolahan data serta pengendalian perangkat IoT.

Pada tahap ini, Node-RED baru berhasil dibuka dan belum dibuat alur komunikasi MQTT. Konfigurasi MQTT Broker dan pengolahan data sensor akan dilakukan pada tahap berikutnya.

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

Skema Rangkaian Perangkat Keras:

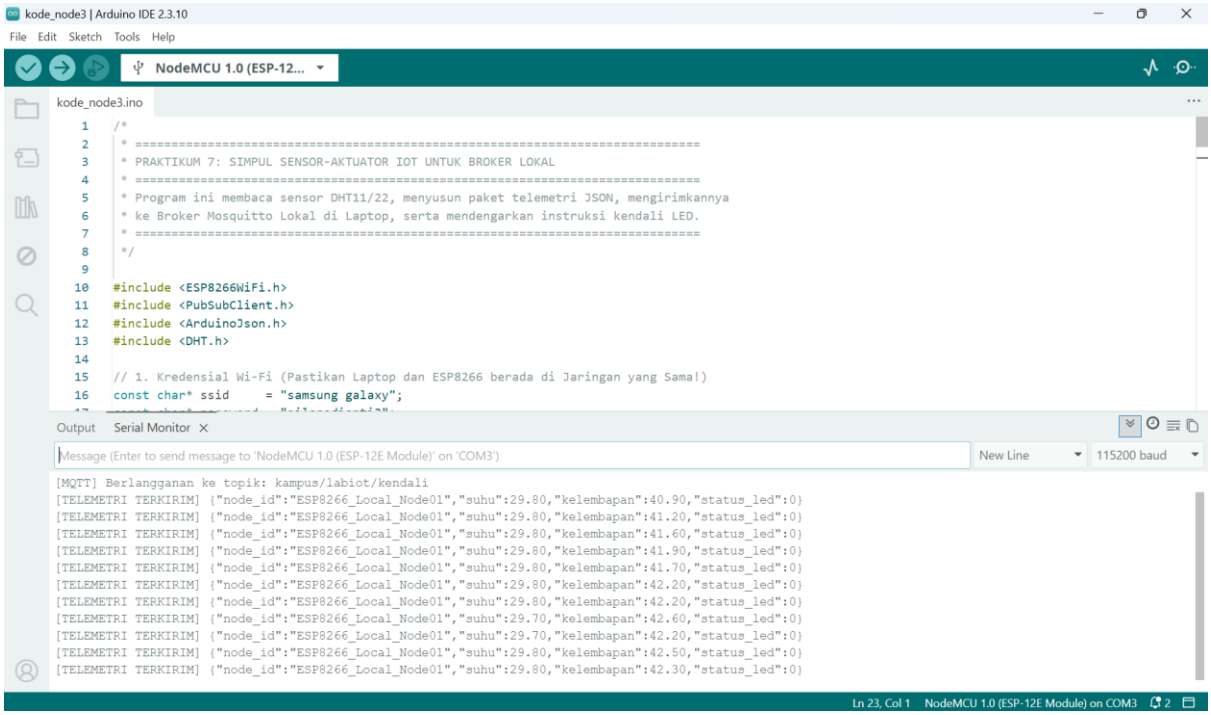
Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

- **Sensor DHT11/DHT22:**
  - Pin VCC →Pin 3V3 NodeMCU
  - Pin GND →Pin GND NodeMCU
  - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
  - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
  - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND

Langkah Pemrograman:

1. Cari tahu **Alamat IPv4 Lokal** laptop Anda yang terhubung ke jaringan Wi-Fi:
   - **Windows:** Buka CMD / PowerShell, ketik ipconfig. Temukan bagian *Wireless LAN adapter Wi-Fi*, lalu catat baris **IPv4 Address** (misalnya: 192.168.1.15).
   - **macOS / Linux:** Buka terminal, ketik ifconfig atau ip a (misalnya: 192.168.1.15).
2. **[PERINGATAN PENTING FIREWALL WINDOWS]:** Sistem keamanan *Windows Defender Firewall* sering kali secara otomatis memblokir koneksi masuk (*inbound connection*) dari port 1883. Jika NodeMCU gagal terhubung ke laptop, buka **Windows Defender Firewall with Advanced Security**, klik **Inbound Rules > New Rule**, pilih **Port > TCP**, masukkan port 1883, lalu pilih **Allow the connection**.
3. Buka **Arduino IDE**, buat berkas sketsa baru, lalu salin kode program terpadu
4. Unggah kode program NodeMCU ESP8266

5. Buka **Serial Monitor** pada kecepatan 115200 baud. Pastikan mikrokontroler berhasil mendapatkan alamat IP dan memunculkan tulisan [MQTT] Menghubungkan ke Broker Lokal... Terhubung! serta mulai mengirimkan payload telemetri JSON secara stabil.



Gambar 5. Hasil Pengiriman Data Sensor NodeMCU ESP8266 melalui MQTT Broker Lokal

Gambar di atas menunjukkan tampilan Serial Monitor pada Arduino IDE setelah program diunggah ke NodeMCU ESP8266. Terlihat bahwa NodeMCU berhasil terhubung ke jaringan Wi-Fi dan MQTT Broker lokal. Selanjutnya, perangkat mengirimkan data sensor berupa suhu, kelembapan, dan status LED dalam format JSON secara berkala. Hasil ini menunjukkan bahwa NodeMCU sudah dapat berkomunikasi dengan MQTT Broker lokal untuk mengirimkan data telemetri.

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
    - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.

- Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
  - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
  - **Name:** Mosquitto Local
  - **Server:** Ketik **mqtt**

    (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
  - **Port:** 1883
  - Klik tombol merah **Add** di pojok kanan atas dialog.

3. **Mengonfigurasi Parameter Topik:**
   - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
   - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
   - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
   - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
   - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**
   - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
   - Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.
6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang ( ) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:
   - kampus/labiot/telemetri : msg.payload : Object
   - {

     node_id: "ESP8266_Local_Node01",

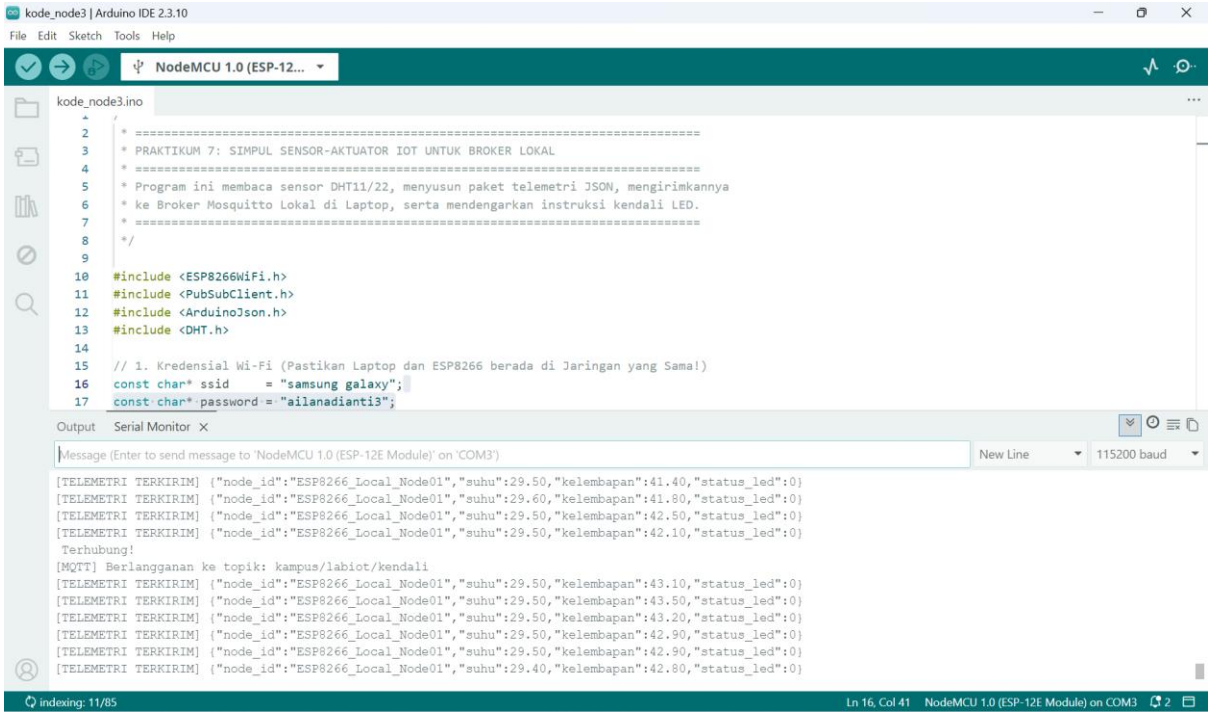
suhu: 29.8,

kelembapan: 62.4,

status_led: 0

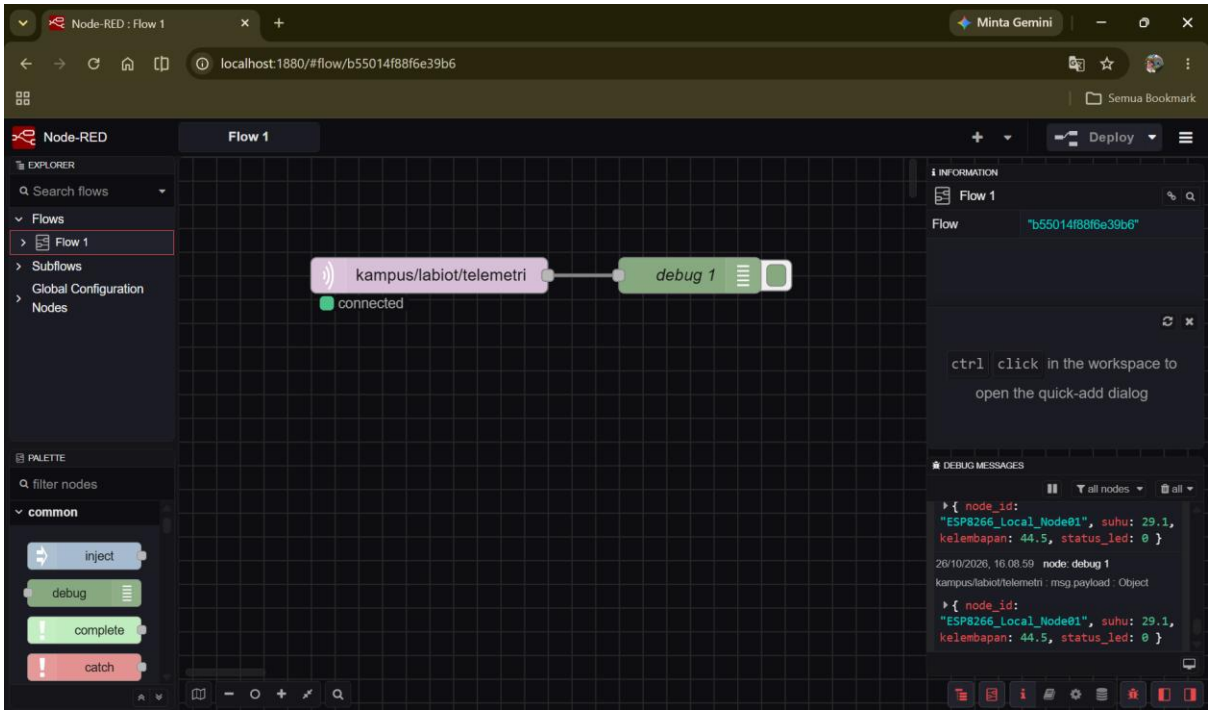
}

- Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.



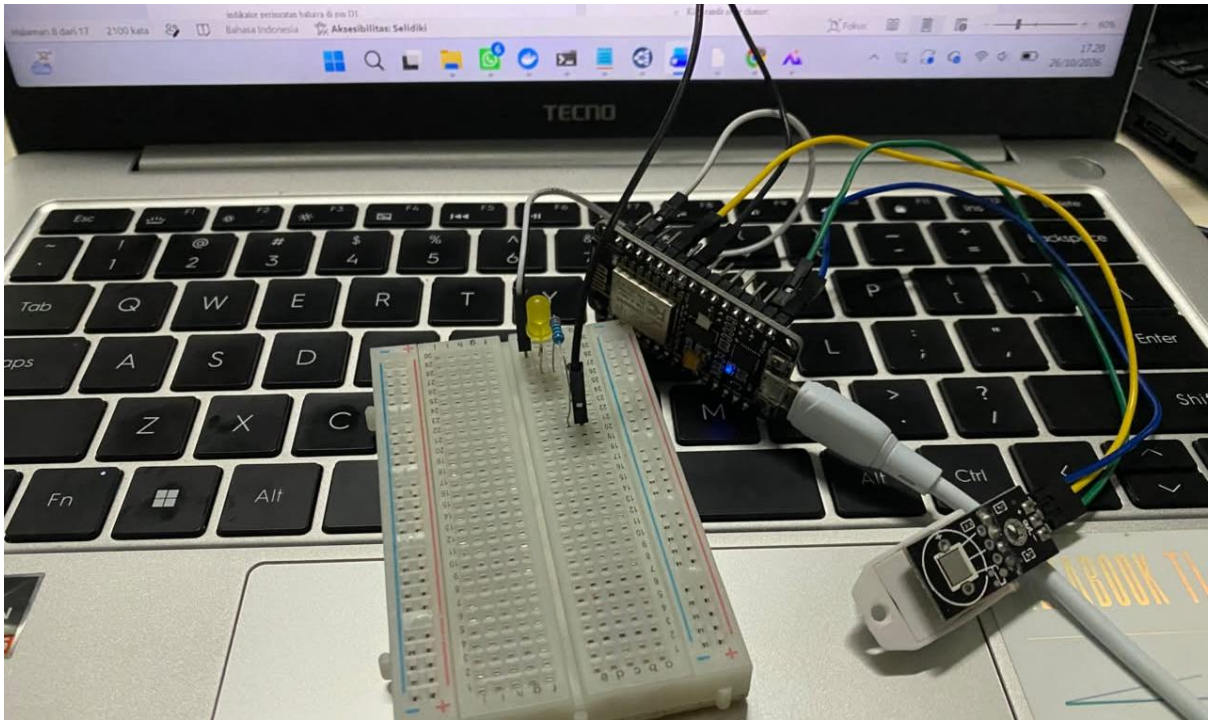
Gambar 6. Konfigurasi MQTT Broker Lokal pada Node-RED

Gambar di atas menunjukkan proses pengaturan MQTT Broker pada node mqtt in di Node-RED. Pada bagian Server, digunakan nama layanan mqtt dengan port 1883. Nama mqtt digunakan karena Node-RED dan Mosquitto berada dalam jaringan Docker Compose yang sama, sehingga keduanya dapat saling terhubung tanpa menggunakan alamat IP laptop.



Gambar 7. Pengaturan Topic MQTT untuk Menerima Data Sensor

Gambar di atas menunjukkan konfigurasi node mqtt in pada Node-RED. Pada bagian Topic, dimasukkan kampus/labiot/telemetri sebagai alamat untuk menerima data yang dikirimkan oleh NodeMCU ESP8266. Bagian Output diatur menjadi *a parsed JSON object* agar data sensor yang diterima dapat langsung dibaca dan diproses oleh Node-RED.



Gambar 8. Rangkaian NodeMCU ESP8266 dengan Sensor DHT dan LED

Gambar di atas menunjukkan rangkaian perangkat IoT yang menggunakan NodeMCU ESP8266, sensor DHT, dan LED pada breadboard. NodeMCU berfungsi sebagai pengendali utama, sensor DHT digunakan untuk membaca suhu dan kelembapan, sedangkan LED berfungsi sebagai indikator keluaran. Rangkaian ini digunakan untuk pengujian sistem pemantauan suhu dan pengendalian LED melalui komunikasi MQTT.
Gambar ini memperlihatkan susunan perangkat keras yang digunakan dalam praktikum, bukan hasil penerimaan data pada panel Debug Node-RED.

7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

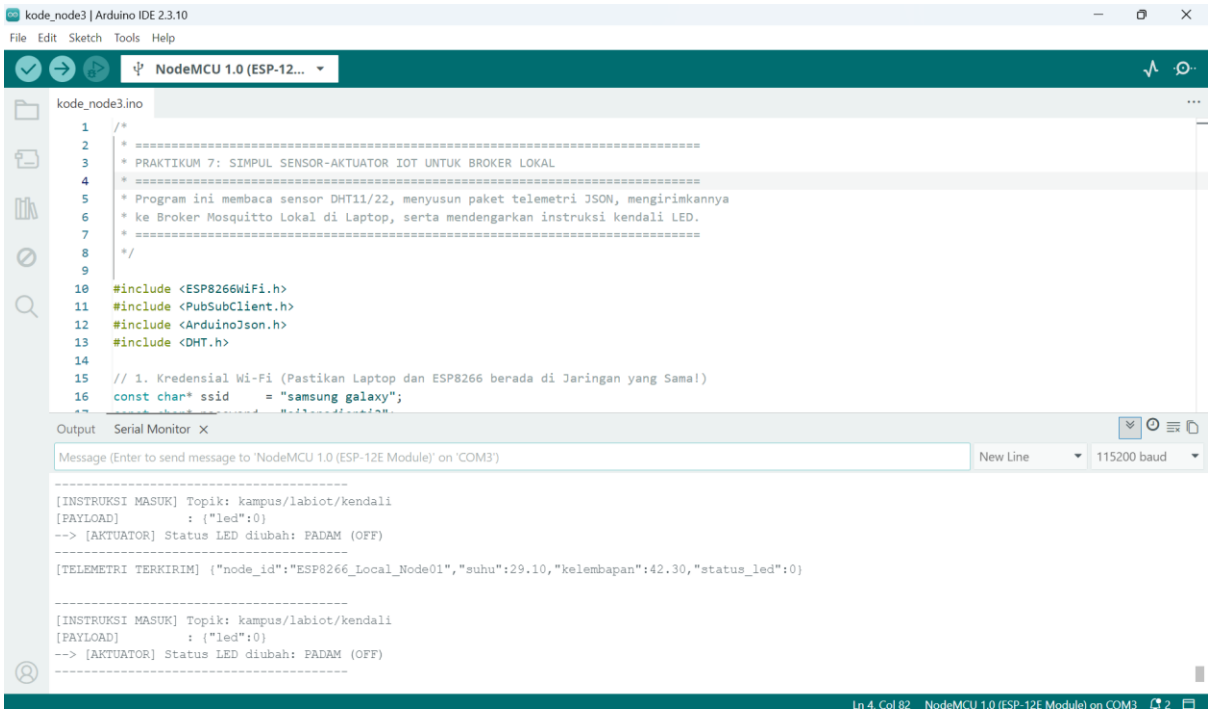
- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.

Langkah Kerja:

1. **Membuat Node Percabangan Logika (switch):**
    - Pada kategori **function**, seret node **switch** ke kanvas.
    - Hubungkan port keluaran node mqtt in ke port masukan node switch.
    - Klik ganda node switch untuk menyetel aturan logika evaluasi:
        - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
        - **Aturan 1 (Kondisi Panas):** Pilih operator **>**, lalu masukkan angka: **31**.
        - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
        - **Aturan 2 (Kondisi Aman):** Pilih operator **<=**, lalu masukkan angka: **31**.
        - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).
2. **Membuat Node Transformasi Muatan Panas (change):**
    - Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
    - Hubungkan port keluaran 1 (suhu > 31) dari node switch ke masukan node change pertama ini.
    - Klik ganda node change:
        - **Name:** Nyalakan LED Indikator

        - **Set:** msg.payload
        - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
        - {
          "led": 1
          }
        - Klik **Done**.
3. **Membuat Node Transformasi Muatan Normal (change):**
    - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
    - Hubungkan port keluaran 2 (suhu ≤ 31) dari node switch ke masukan node change kedua ini.
    - Klik ganda node change:
        - **Name:** Padamkan LED Indikator
        - **Set:** msg.payload
        - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:
        - {
          "led": 0
          }
        - Klik **Done**.
4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
    - Pada kategori **network**, seret node **mqtt out** ke kanvas.
    - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
    - Klik ganda node mqtt out:
        - **Server:** Pilih Mosquitto Local.
        - **Topic:** kampus/labiot/kendali
        - **QoS:** 0 (atau 1)
        - **Retain:** false
        - Klik **Done**.
5. **Menerapkan Perubahan (*Deploy*):**
    - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**

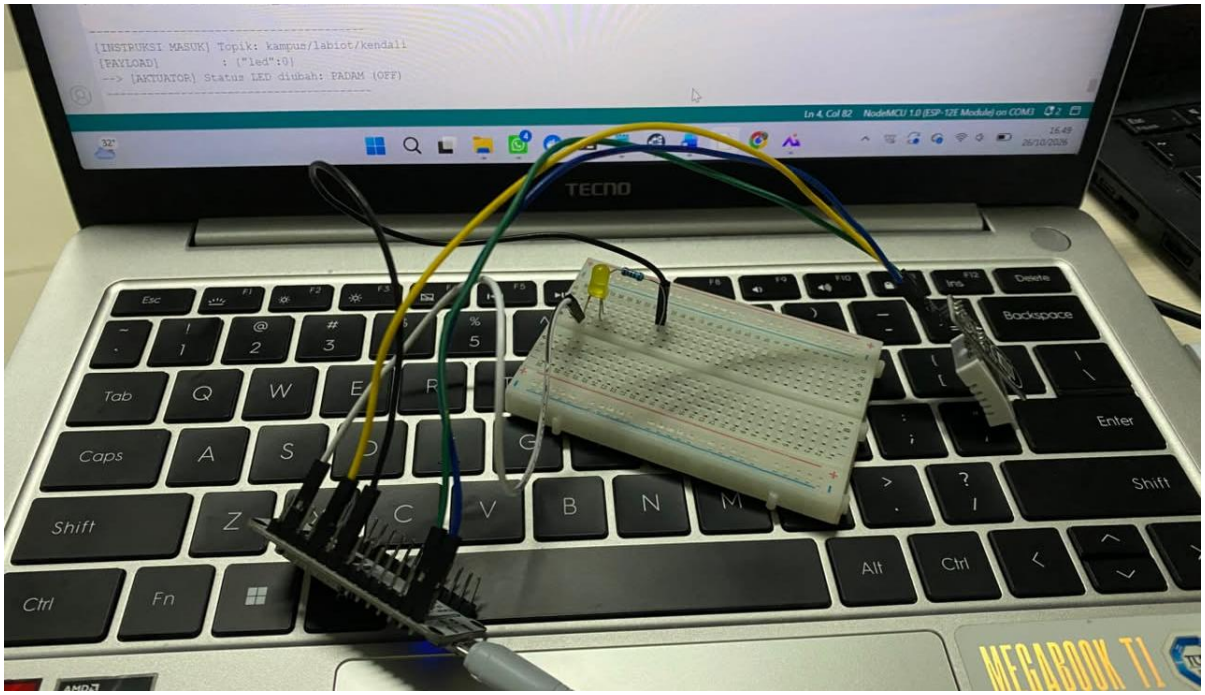
- Amati status awal: Karena suhu ruangan umumnya berkisar antara 26°C – 29°C(≤ 31°C), LED indikator berada dalam kondisi padam.
- Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0°C.
- **Hasil Pengamatan:** Seketika suhu melampaui 31.0°C , Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
- Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0°C. LED indikator akan padam kembali secara otomatis.



Gambar 9. Hasil Pemantauan Data Sensor melalui Serial Monitor Arduino IDE

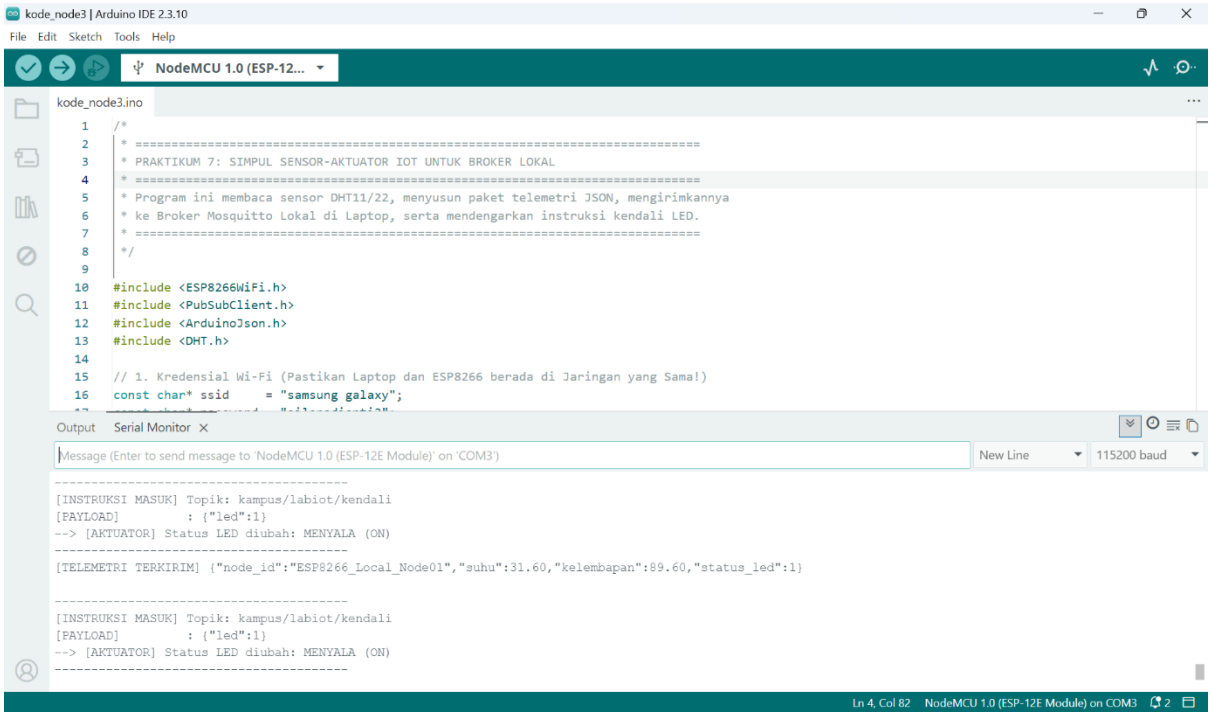
Gambar di atas menunjukkan tampilan Serial Monitor pada Arduino IDE yang digunakan untuk memantau aktivitas NodeMCU ESP8266. Pada Serial Monitor terlihat informasi komunikasi MQTT dan pengiriman data sensor. Data tersebut digunakan untuk memeriksa apakah NodeMCU dapat mengirimkan informasi suhu, kelembapan, dan status LED ke MQTT Broker lokal.

Pada gambar, LED terlihat tidak menyala dan Serial Monitor menampilkan status PADAM (OFF). Jadi, foto ini merupakan dokumentasi hasil pengujian pengendalian LED, bukan pengaturan Node-RED.



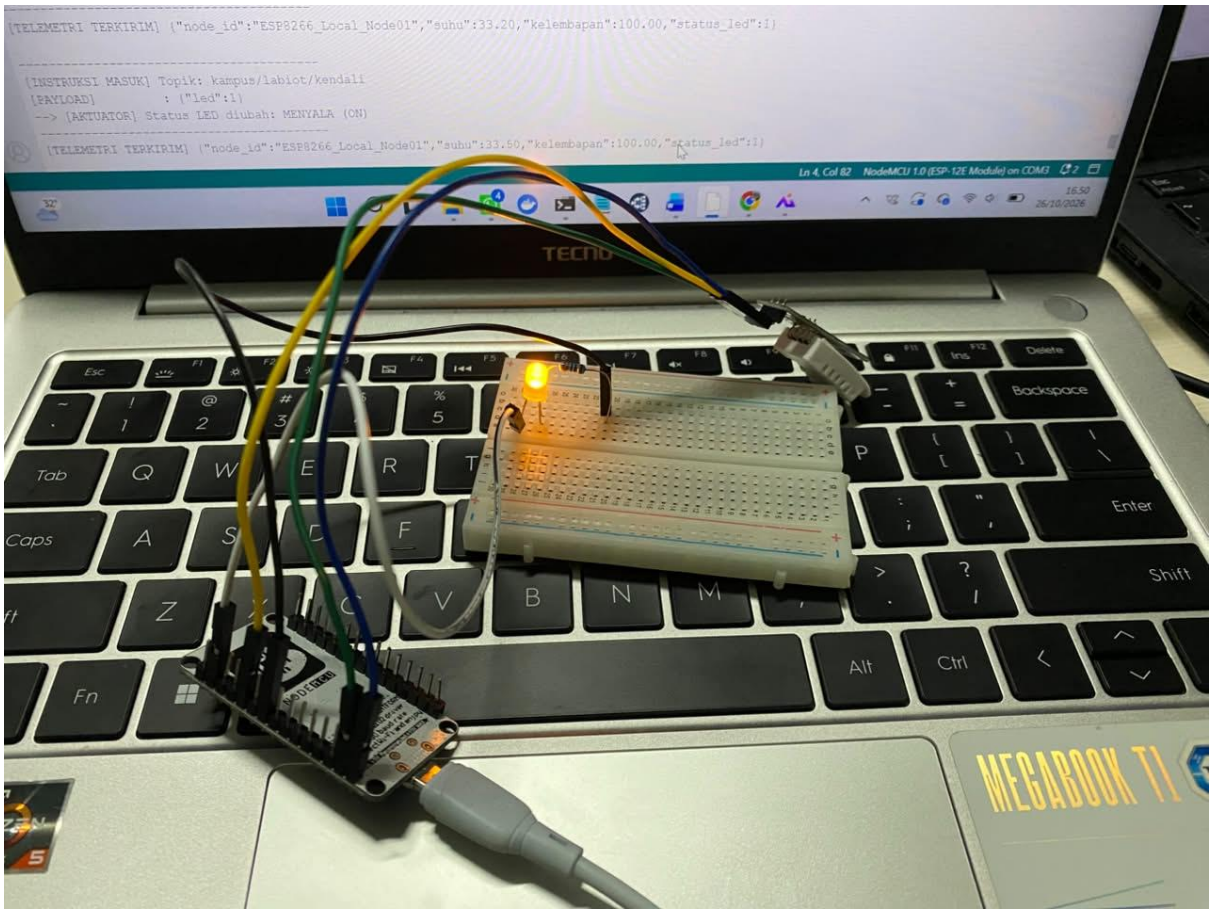
Gambar 10. Hasil Pengujian LED dalam Kondisi Mati (OFF)

Gambar di atas menunjukkan rangkaian NodeMCU ESP8266 yang terhubung dengan LED pada breadboard. Terlihat bahwa LED berada dalam keadaan mati (OFF). Pada layar laptop juga terlihat hasil Serial Monitor yang menampilkan pesan bahwa status LED diubah menjadi PADAM (OFF) setelah menerima perintah melalui topik MQTT kampus/labiot/kendali. Hal ini menunjukkan bahwa NodeMCU telah menerima dan menjalankan perintah untuk mematikan LED.



Gambar 11. Hasil Pengujian Pengendalian LED pada Arduino IDE

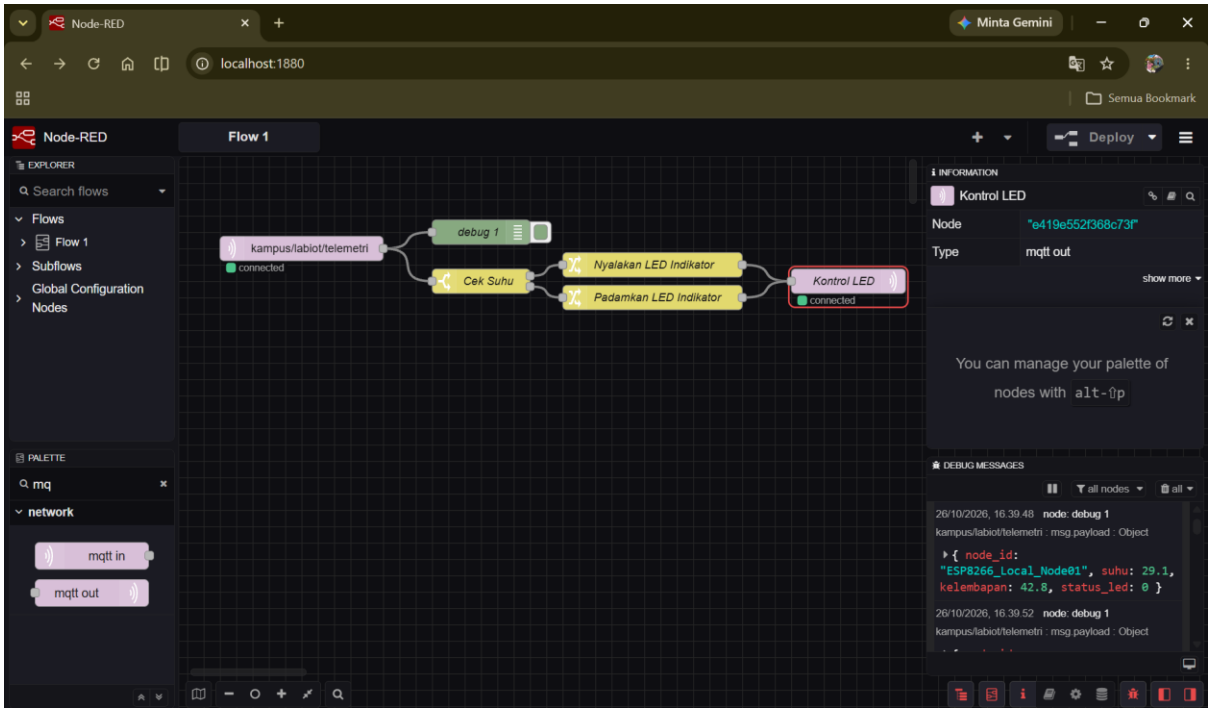
Gambar di atas menunjukkan tampilan Serial Monitor pada Arduino IDE yang digunakan untuk memantau perintah pengendalian LED pada NodeMCU ESP8266. Pada Serial Monitor terlihat informasi mengenai instruksi MQTT yang diterima melalui topik kampus/labiot/kendali serta perubahan status LED. Pengujian ini digunakan untuk memastikan bahwa NodeMCU dapat menerima dan memproses perintah pengendalian LED dari MQTT Broker lokal.



Gambar 12. Hasil Pengujian LED dalam Kondisi Menyala (ON)

Gambar di atas menunjukkan rangkaian NodeMCU ESP8266 yang terhubung dengan LED pada breadboard. Terlihat bahwa LED berwarna kuning sedang menyala. Kondisi ini menunjukkan bahwa LED telah diaktifkan oleh NodeMCU. Pengujian ini digunakan untuk mendokumentasikan kondisi LED menyala sebagai bagian dari sistem pengendalian perangkat IoT.

Pada foto terlihat LED menyala dengan jelas. Namun, foto ini saja belum menunjukkan nilai suhu yang menyebabkan LED menyala, sehingga keterangannya difokuskan pada kondisi LED yang terlihat.



Gambar 13. Hasil Pembuatan Flow Pengendalian LED Berdasarkan Suhu pada Node-RED

Gambar di atas menunjukkan tampilan flow Node-RED yang digunakan untuk mengendalikan LED berdasarkan data suhu dari NodeMCU ESP8266. Terlihat node mqtt in dengan topik kampus/labiot/telemetri yang terhubung ke node debug dan Cek Suhu. Selanjutnya, terdapat node Nyalakan LED Indikator dan Padamkan LED Indikator yang terhubung ke node Kontrol LED. Pada panel Debug di sebelah kanan juga terlihat data telemetri yang diterima dari NodeMCU. Flow ini digunakan untuk mengatur pengiriman perintah LED berdasarkan kondisi suhu.

Gambar ini memperlihatkan susunan node dan data yang masuk pada Node-RED. Foto ini bukan pengujian fisik LED, melainkan tampilan alur pengendalian yang dibuat.

7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):** Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

   Jawab: Karena localhost itu mengarah ke container Node-RED itu sendiri, bukan ke MQTT. Jadi, Node-RED tidak bisa terhubung ke MQTT kalau menggunakan localhost.

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

   Jawab: Docker DNS membantu mencari alamat container lewat nama layanannya, misalnya mqtt. Sedangkan Bridge Network menghubungkan container supaya bisa saling berkomunikasi dalam satu jaringan.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):** Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

     Jawab: Karena IP NodeMCU dan laptop berada di jaringan yang berbeda. Jadi, keduanya tidak bisa langsung terhubung kalau tidak ada jalur penghubung antarjaringan.

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

     Jawab: Caranya dengan menghubungkan laptop dan NodeMCU ke Wi-Fi atau hotspot yang sama. Setelah itu, cek IP keduanya dan pastikan berada di jaringan yang sama. Kalau masih gagal, periksa firewall laptop.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**

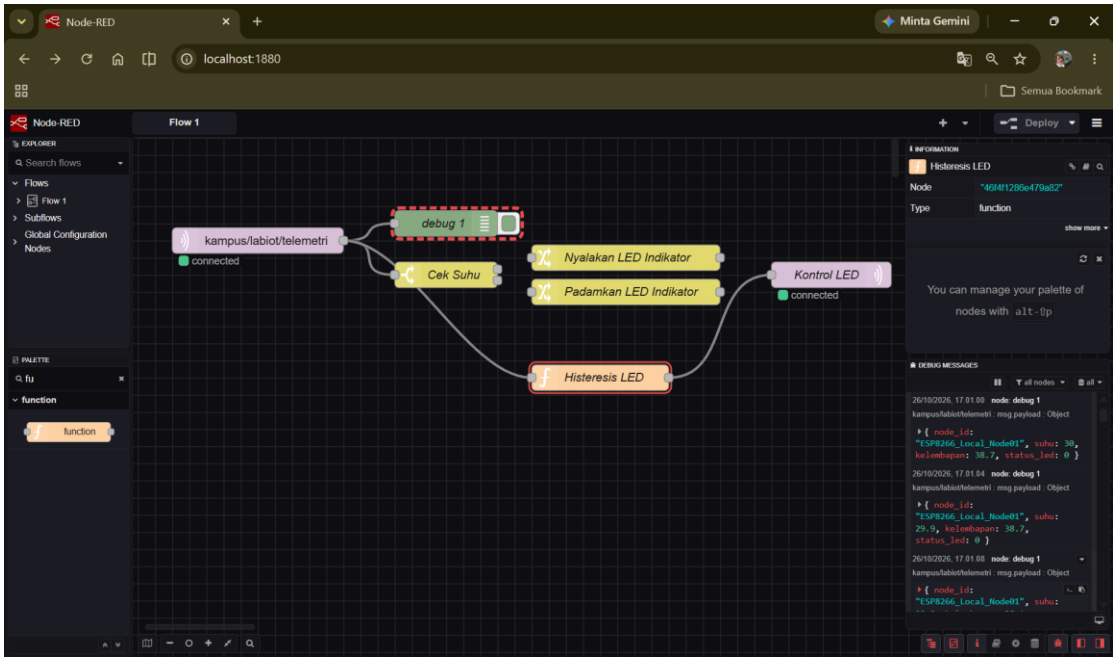
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas $31.0°C$(misal: $31.05°C \rightarrow 30.95°C \rightarrow 31.02°C$), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.

  Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

  - LED baru menyala jika suhu $> 31.5°C$(msg.payload = {"led": 1}).

- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).

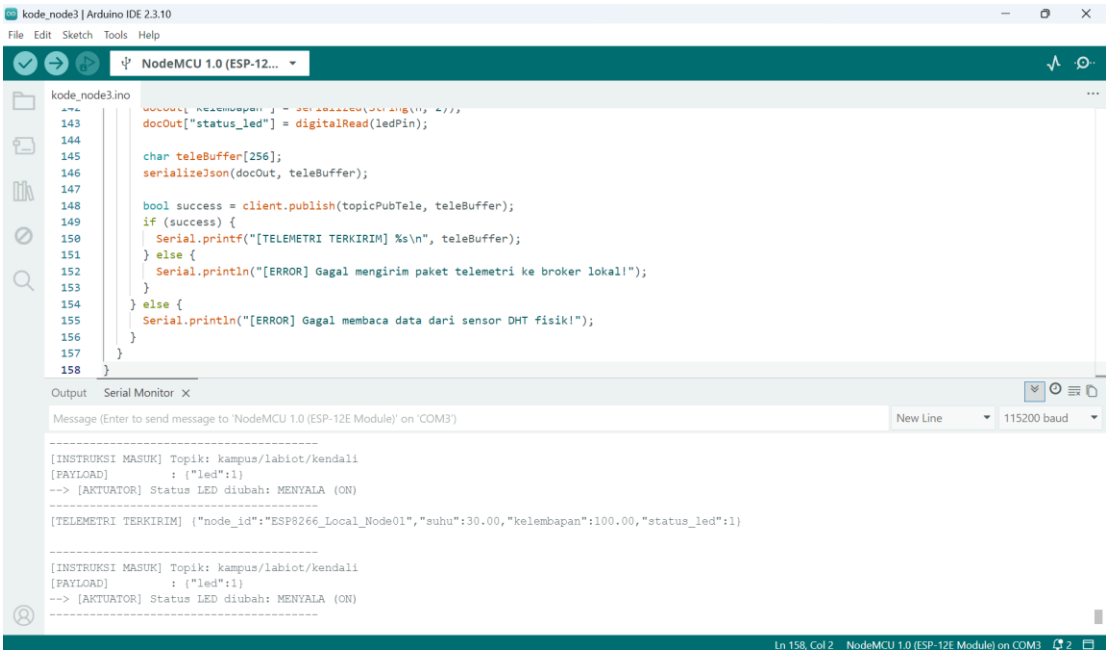
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!



Gambar 14. Implementasi Function Node Histeresis pada Node-RED

Gambar di atas menunjukkan tampilan flow Node-RED yang telah ditambahkan node Histeresis LED. Terlihat bahwa node mqtt in menerima data telemetri dari NodeMCU ESP8266, kemudian meneruskan data tersebut ke node Histeresis LED untuk diproses. Hasil pengolahan selanjutnya dikirimkan melalui node Kontrol LED untuk mengatur kondisi LED. Pada bagian kanan juga terlihat panel Debug yang menampilkan data sensor yang diterima.

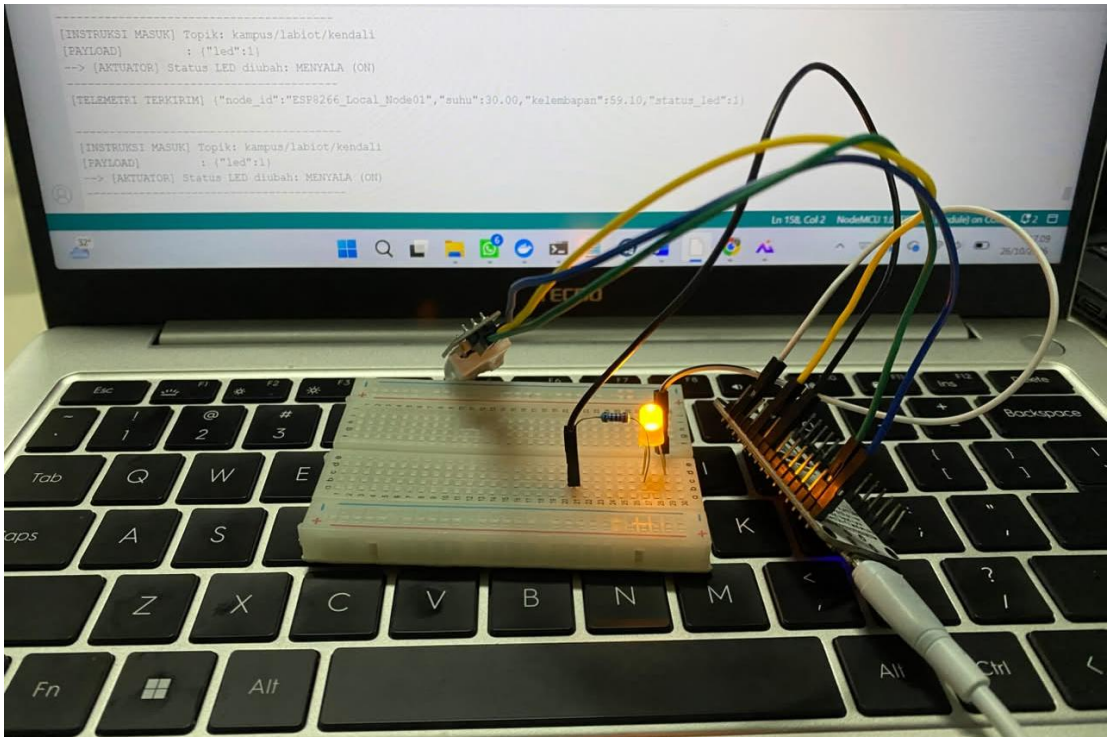
Pada gambar, node Histeresis LED sudah terhubung langsung ke Kontrol LED, sedangkan node Nyalakan LED Indikator dan Padamkan LED Indikator masih terlihat tetapi tidak terhubung ke node keluaran. Artinya, jalur pengendalian LED yang digunakan pada flow tersebut adalah melalui node histeresis.



Gambar 15. Hasil Penerimaan Perintah LED dan Pengiriman Data Telemetri pada NodeMCU ESP8266

Gambar di atas menunjukkan hasil pengujian komunikasi MQTT melalui Serial Monitor pada Arduino IDE. Terlihat bahwa NodeMCU ESP8266 menerima perintah {"led":1} melalui topik kampus/labiot/kendali, kemudian mengubah status LED menjadi menyala (ON). Selain itu, Serial Monitor juga menampilkan data telemetri yang berhasil dikirim, berupa suhu 30°C, kelembapan 100%, dan status LED bernilai 1. Hasil ini menunjukkan bahwa NodeMCU dapat menerima perintah pengendalian LED sekaligus mengirimkan data sensor melalui MQTT Broker lokal.

Tulisan [TELEMETRI TERKIRIM] menunjukkan bahwa proses pengiriman data berhasil dilakukan oleh program. Sementara itu, tulisan Status LED diubah: MENYALA (ON) menunjukkan bahwa NodeMCU telah memproses perintah untuk menyalakan LED.



Gambar 16. Hasil Pengujian LED dalam Kondisi Menyala (ON)

Gambar di atas menunjukkan rangkaian perangkat IoT yang terdiri dari NodeMCU ESP8266, breadboard, sensor DHT, dan LED. Terlihat bahwa LED berwarna kuning sedang menyala (ON). Kondisi ini memperlihatkan hasil pengujian perangkat keras setelah NodeMCU menjalankan perintah untuk mengaktifkan LED. Pengujian ini digunakan untuk memastikan bahwa LED dapat berfungsi sebagai indikator keluaran pada sistem IoT.

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**

  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

     Jawab: Kalau volume dihapus, data Node-RED hanya tersimpan di dalam container. Saat menjalankan docker compose down, container akan dihapus. Jadi, ketika dijalankan lagi, flow atau alur yang sudah kita buat bisa hilang dan harus dibuat ulang.

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Jawab: Bind Mount adalah penyimpanan data di folder laptop yang kita tentukan sendiri, misalnya D:\iot-server\nodered. Jadi, kita bisa langsung melihat dan mengelola filenya.

Sedangkan Named Volume adalah penyimpanan yang tempatnya diatur langsung oleh Docker, jadi kita tidak perlu menentukan foldernya sendiri.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:

docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: **rahasia123** (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.
- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

- Ubah nilai allow_anonymous true menjadi:

allow_anonymous false

- Tambahkan baris pemanggilan berkas kata sandi:

password_file /mosquitto/config/password.txt

- Simpan berkas mosquitto.conf.

3. Memuat Ulang Layanan Broker:

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:

docker restart mqtt-broker

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan **disconnected** atau **connection refused**.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:
- [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...

(Catatan: Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized').

5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

1. **Pemutakhiran Skrip Program NodeMCU ESP8266:**
    - Buka sketsa program Arduino IDE dari Praktikum 2.
    - Modifikasi bagian pemanggilan fungsi client.connect() di dalam fungsi reconnectMQTT() dengan menyertakan kredensial resmi:

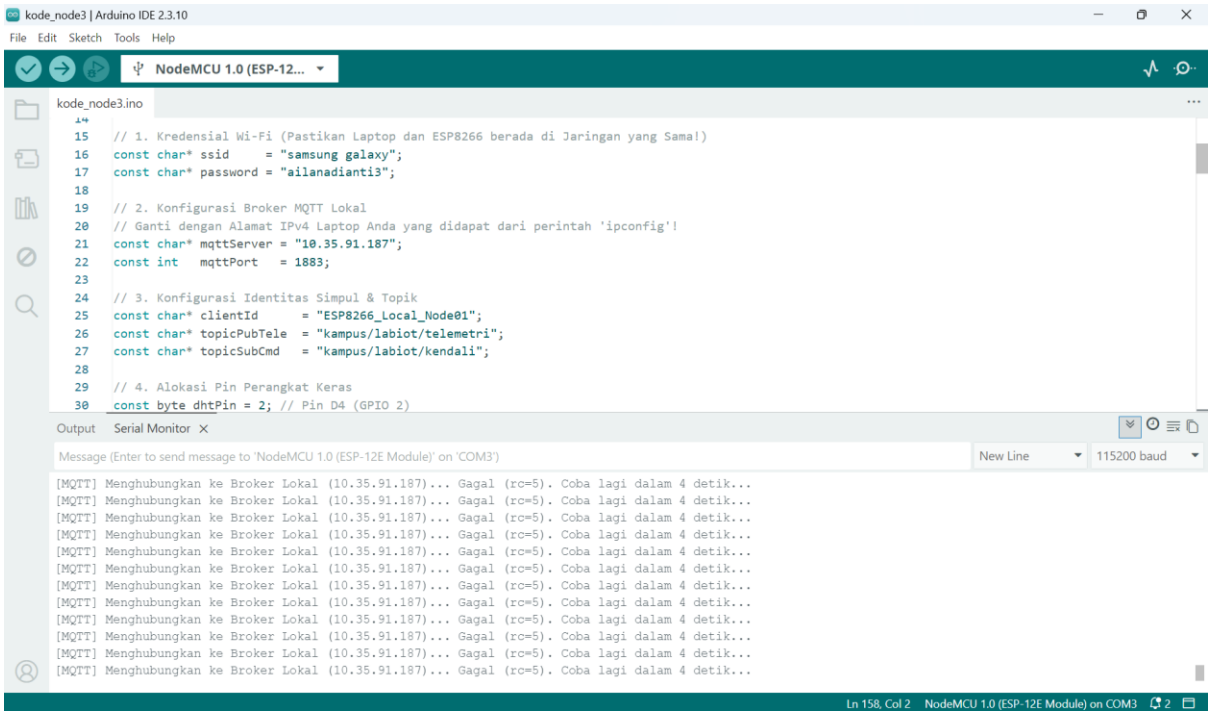
      ```
      // Format: client.connect(clientId, username, password)
      if (client.connect(clientId, "admin", "rahasia123")) {
        Serial.println(" Terhubung dengan Autentikasi Sah!");
        client.subscribe(topicSubCmd);
      }
      ```

    - Unggah ulang kode ke NodeMCU dan amati bahwa ESP8266 kini kembali berhasil terhubung (*rc=0*).
2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**
    - Pada kanvas Node-RED, klik ganda node mqtt in.
    - Klik ikon pensil ( ) di samping nama broker Mosquitto Local.
    - Buka tab **Security** di bagian atas dialog konfigurasi.
    - Masukkan **Username:** admin dan **Password:** rahasia123.
    - Klik tombol **Update**, lalu klik **Done**, dan tekan tombol merah **Deploy**.
    - Amati bahwa indikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar!

Kriteria Keberhasilan:

- Broker Mosquitto terbukti menolak akses anonim tanpa kredensial (allow_anonymous false).

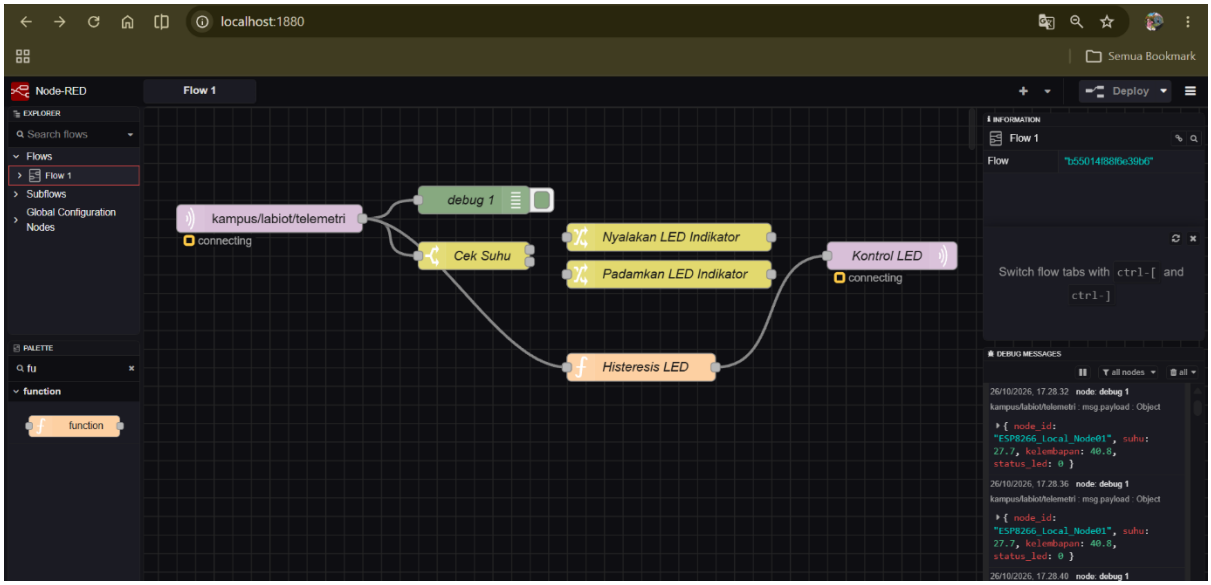
- Terdapat berkas password.txt yang memuat enkripsi kata sandi akun pengguna.
- Serial Monitor NodeMCU membuktikan pemulihan koneksi sukses menggunakan kredensial autentikasi resmi.
- Seluruh aliran kerja (*flow*) pada Node-RED kembali terhubung dan mampu memicu otomasi LED secara stabil.



Gambar 17. Hasil Pengujian Koneksi NodeMCU Setelah Autentikasi MQTT Diaktifkan

Gambar di atas menunjukkan tampilan Arduino IDE dan Serial Monitor saat NodeMCU ESP8266 mencoba terhubung ke MQTT Broker lokal. Pada Serial Monitor terlihat pesan kegagalan koneksi yang muncul secara berulang setelah sistem autentikasi MQTT diaktifkan. Kondisi ini terjadi karena program NodeMCU belum menggunakan username dan password yang diperlukan oleh MQTT Broker. Oleh karena itu, kode program perlu diperbarui dengan menambahkan kredensial autentikasi agar perangkat dapat terhubung kembali.

Gambar ini menunjukkan tahap pengamatan masalah pada bagian 7.7, yaitu setelah MQTT Broker mewajibkan autentikasi. Ini bukan foto pengujian LED, melainkan dokumentasi kegagalan koneksi sebelum kode NodeMCU diperbaiki.



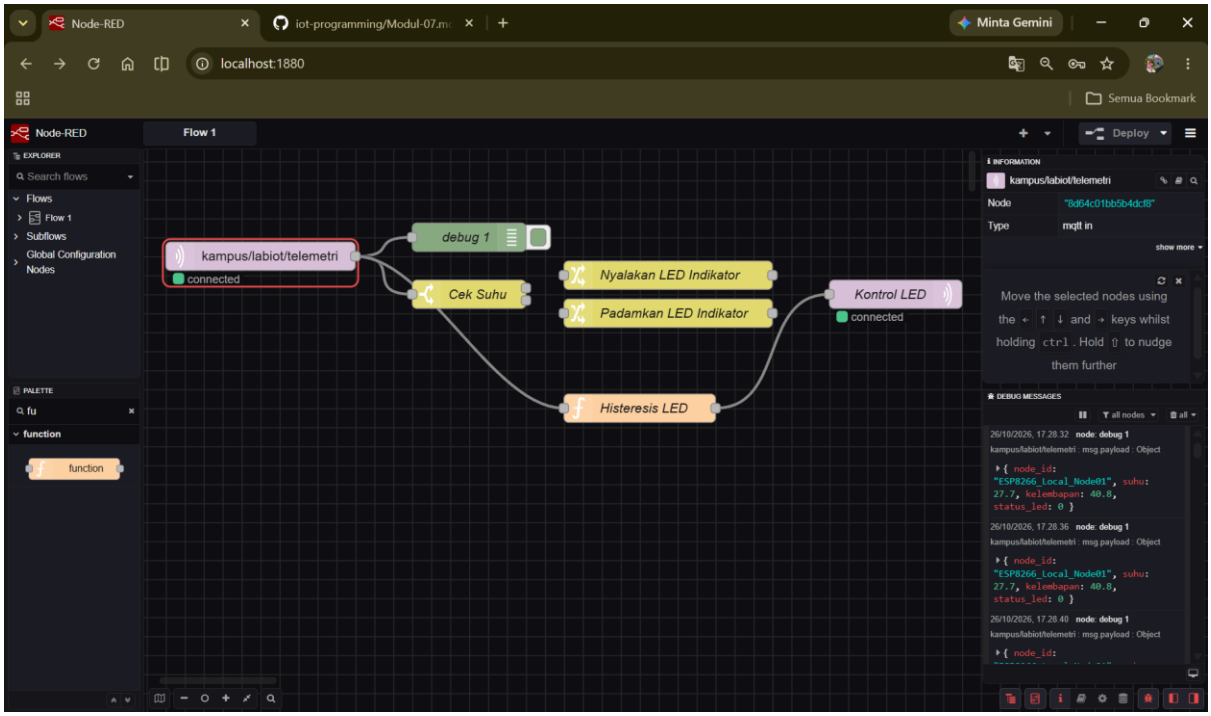
Gambar 18. Status Koneksi MQTT pada Node-RED Setelah Autentikasi Diaktifkan

Gambar di atas menunjukkan tampilan flow Node-RED setelah sistem keamanan pada MQTT Broker diaktifkan. Terlihat bahwa node penerima dan pengirim MQTT memiliki indikator berwarna kuning dengan status *connecting*. Kondisi ini menunjukkan bahwa Node-RED sedang mencoba terhubung kembali ke MQTT Broker, tetapi koneksi belum berhasil. Pada tahap ini, username dan password belum dimasukkan ke konfigurasi broker pada Node-RED, sehingga diperlukan pengaturan autentikasi agar komunikasi data dapat berjalan kembali.



Gambar 19. Penambahan Username dan Password MQTT pada Program NodeMCU ESP8266

Gambar di atas menunjukkan perubahan kode program NodeMCU ESP8266 pada Arduino IDE, khususnya bagian fungsi reconnectMQTT(). Pada bagian client.connect(), ditambahkan username dan password agar NodeMCU dapat melakukan autentikasi saat terhubung ke MQTT Broker. Perubahan ini dilakukan karena MQTT Broker sudah menggunakan sistem keamanan yang mewajibkan setiap perangkat memasukkan kredensial yang sesuai. Dengan demikian, NodeMCU dapat mencoba terhubung kembali menggunakan akun yang telah didaftarkan.



Gambar 20. Hasil Pemulihan Koneksi MQTT pada Node-RED

Gambar di atas menunjukkan tampilan Node-RED setelah konfigurasi autentikasi MQTT diperbarui. Terlihat bahwa node mqtt in dan mqtt out sudah memiliki indikator hijau dengan status *connected*. Hal ini menunjukkan bahwa Node-RED berhasil terhubung kembali ke MQTT Broker setelah menggunakan username dan password yang sesuai. Pada panel Debug juga terlihat data telemetri berupa suhu, kelembapan, dan status LED yang diterima dari NodeMCU ESP8266. Hasil ini menunjukkan bahwa komunikasi data melalui MQTT Broker sudah kembali berjalan.